

Team Number:	apmcm253XXXXX
Problem Chosen:	A

2025 APMCM summary sheet

Methodology: We first formulate a classical mixed-integer Unit Commitment (UC) model, extend it to a network- and security-constrained variant, translate the classical constraints into a Quadratic Unconstrained Binary Optimization (QUBO) representation compatible with the Kaiwu SDK, and finally design a scale-reduction strategy that compresses the QUBO so it fits current quantum hardware.

Results: The classical model solves in 0.03 s with an optimal cost of 15,129.08 \$, the networked formulation adds 196,900 constraints yet maintains the same cost while exposing congestion patterns, the QUBO contains 7,024 bits with density 6.6×10^{-4} and produces heuristic schedules after simulated annealing, and the reduced QUBO shrinks to 435 bits, reaches a Hamiltonian of $-16,486$, and achieves a cost of 26,775.55 \$ on real hardware logs.

Conclusion: Combining classical optimization with quantum formulations reveals where security requirements drive dispatch changes, quantifies current quantum limitations, and demonstrates a viable reduction pipeline that keeps the UC physics interpretable while preparing problem instances for near-term devices.

Keywords: Unit Commitment Security-Constrained UC QUBO Quantum Optimization

Contents

1. Introduction.....	1
1.1 Problem Background.....	1
1.2 Problem Restatement.....	1
1.3 Problem Analysis	2
2. Assumptions.....	2
3. Key Results Summary.....	3
4. Notations	4
5. Problem 1: Classical Unit Commitment.....	6
5.1 Model description	6
5.1.1 <i>Objective</i>	6
5.1.2 <i>Power balance and capacity</i>	6
5.1.3 <i>Logic and minimum-time constraints</i>	6
5.1.4 <i>Ramp rate</i>	7
5.2 Solution workflow	7
5.3 Solver performance	7
5.4 Dispatch analysis.....	7
6. Problem 2: Network and Security Constrained UC	8
6.1 Model extensions.....	8
6.1.1 <i>Flexible load modeling</i>	8
6.1.2 <i>DC power flow</i>	9
6.1.3 <i>Spinning reserve and response</i>	10
6.1.4 <i>N-1 contingencies</i>	10
6.2 Solution workflow	11
6.3 Results and analysis.....	11
7. Problem 3: QUBO Transformation and Quantum Solution	12
7.1 Binary encoding and penalties	12
7.2 Kaiwu workflow	13
7.3 Result comparison.....	14
8. Problem 4: Scale Reduction and Feasibility.....	16
8.1 Reduction strategy.....	16
8.2 Results	16

9. Strengths and Weaknesses	17
9.1 Strengths of Our Approach	17
9.2 Weaknesses and Limitations	17
9.3 Areas for Improvement	17
10. References	18
11. Appendix	19
11.1 Key Implementation Code Snippets	19
11.1.1 <i>Classical UC Model Implementation</i>	19
11.1.2 <i>QUBO Transformation Code</i>	20

I. Introduction

1.1 Problem Background

Unit Commitment (UC) is a fundamental optimization problem in power system operation that determines the on/off status and generation levels of thermal units over a given time horizon. The primary goal is to minimize operating costs while satisfying demand and various system constraints. The UC problem is known to be a large-scale mixed integer programming (MIP) challenge, whose computational complexity grows exponentially with system size.

With the emergence of quantum optimization technologies such as the Coherent Ising Machine (CIM), the UC problem can be reformulated as a Quadratic Unconstrained Binary Optimization (QUBO) model. This transformation enables the use of quantum-inspired solvers for fast and parallel search of optimal solutions, paving the way for real-time power system operation on quantum hardware.

This competition invites participants to bridge classical power system optimization with quantum computing through a three-stage workflow: building a classical unit commitment model, transforming it into a QUBO formulation, and designing a scalable reduction strategy to accommodate current quantum hardware constraints.

1.2 Problem Restatement

Participants are required to construct a complete UC optimization framework that integrates both classical and quantum optimization methods. The problem consists of four sequential tasks:

- 1. Classical UC Modeling and Optimization:** Formulate and solve a classical Unit Commitment model using conventional optimization tools (e.g., Gurobi or CPLEX), including fuel costs, startup/shutdown costs, power balance constraints, generation limits, minimum up/down time constraints, ramp-rate constraints, and startup/shutdown logic constraints.
- 2. UC Modeling with Network and Security Constraints:** Extend the classical UC model to include network power flow constraints, N-1 security constraints, spinning reserve requirements, and optionally minimum safety inertia constraints.
- 3. QUBO Transformation and Quantum Solution:** Transform the UC model into a QUBO representation and solve it using the Kaiwu SDK, comparing quantum-

based solutions with traditional optimization results.

4. **Problem Scale Reduction under Quantum Hardware Constraints:** Design and implement an effective problem-scale reduction strategy specifically for the UC model to ensure the resulting QUBO can be successfully solved under the bit-capacity limitation of current quantum hardware.

1.3 Problem Analysis

The UC problem presents significant computational challenges due to its combinatorial nature and large-scale characteristics. The classical approach involves solving a complex MIP problem with numerous constraints, while the quantum approach requires careful transformation of the problem into a QUBO format suitable for quantum solvers.

Key challenges include:

- **Combinatorial Complexity:** The binary decision variables for unit commitment status create an exponentially large solution space.
- **Constraint Handling:** Multiple operational constraints must be satisfied simultaneously, including technical limits and security requirements.
- **Model Transformation:** Converting the classical UC model into a QUBO formulation requires careful penalty parameter selection and constraint relaxation.
- **Scale Reduction:** Current quantum hardware limitations necessitate effective problem reduction strategies to make real-world UC problems tractable.
- **Solution Quality:** Ensuring that quantum solutions maintain feasibility and optimality comparable to classical approaches.

This paper addresses these challenges through a systematic approach that combines classical optimization techniques with emerging quantum computing methods, providing insights into the practical implementation of quantum optimization for power system applications.

II. Assumptions

To simplify the complex Unit Commitment problem and make it tractable for both classical and quantum optimization approaches, we make the following assumptions:

1. **Time Horizon Limitation:** The optimization is conducted over a 24-hour period only, focusing on short-term operational planning rather than long-term strategic decisions.

2. **Minimum Power Generation:** We assume that the minimum power generation values from Table 2 represent the true minimum operating limits for each unit, as Table 1 also contains minimum power values that may be inconsistent.
3. **Initial Operating Conditions:** If a unit is online at time 0, we assume it operates at its minimum power output level to establish a realistic starting condition for the optimization.
4. **Network Parameter Simplification:** We ignore the resistance (r) and susceptance (b) parameters from Table 3, focusing only on the reactance (x) values for DC power flow calculations, as these are the primary parameters affecting power flow distribution.
5. **Bus-Level Load Distribution:** Since the problem does not specify individual bus-level load demands, we treat bus-level load demands as optimization variables in Problem 2, allowing flexible load allocation across the network.
6. **Inertia Constraint Simplification:** Due to the unclear relationship between minimum inertia requirements (H_{min}) and unit inertia constants (H), we do not consider inertia constraints in the optimization process, focusing instead on the core UC constraints.
7. **Deterministic Load Forecasting:** We assume perfect load forecasting for the 24-hour period, with no uncertainty in the load demand values provided in Table 4.
8. **Constant Fuel Prices:** Fuel prices and cost coefficients remain constant throughout the optimization horizon, with no consideration for fuel price volatility.
9. **No Transmission Losses:** We neglect transmission losses in the power system network, assuming that all generated power reaches the load centers without losses.
10. **Independent Unit Failures:** For N-1 security analysis, we assume that unit and line failures occur independently and that only single contingencies need to be considered.

III. Key Results Summary

We summarize the most important outcomes immediately after the introduction so that the reader can map each contribution to the subsequent sections.

- **Problem 1 — Classical UC:** Gurobi solves the 1,156-row model with 0.009% optimality gap and 0.03 s runtime (Table 2). Representative dispatch snapshots in Table 3 and Figures 1-2 show the expected base-/mid-/peak-load roles.

- **Problem 2 — Network and Security Constrained UC:** Introducing flexible load allocation, DC flow and N-1 scenarios enlarges the matrix to 196,900 rows but presolve compresses it to 43,262 (Table 4). Costs remain at 15,129.08 \$ while Figures 3-8 expose the spatial load transfers, outage reactions and reserve burden.
- **Problem 3 — QUBO and Kaiwu:** The full QUBO reaches 7,024 binary variables with density 6.6×10^{-4} and energy -3473 after simulated annealing. Table 5 and Figure 9 compare the quantum dispatch with the classical baseline and highlight constraint violations, providing concrete guidance on penalty tuning.
- **Problem 4 — Scale Reduction:** By trimming slack variables and compressing encoded ranges, the bit width drops to 435 and Hamiltonian trajectories in Figure 10 confirm stable convergence to $-16,486$. The resulting schedule costs 26,775.55 \$ and proves that the reduced QUBO can run on real hardware logs from Kaiwu.

IV. Notations

The notation in Table 1 consolidates all symbols that appear in Problems 1–4 as well as in the QUBO reformulation. A nonfloating longtable is used to guarantee that the notation summary is rendered directly under the section title.

Table 1 Mathematical notation summary

Symbol	Description	Problems
Sets and indices		
$\mathcal{I}$	Thermal unit set	Q1–Q4
$\mathcal{B}$	Bus (node) set	Q2
$\mathcal{L}$	Transmission line set	Q2
$\mathcal{T}$	= Scheduling horizon	Q1–Q4
$\{1, \dots, 24\}$		
C^G, C^L	Generator and line contingency sets	Q2
i, b, ℓ, t	Unit, bus, line, and time indices	Q1–Q4
Parameters		
$P_i^{\min}, P_i^{\max}$	Minimum/maximum power of unit i	Q1–Q4
a_i, b_i, c_i	Quadratic fuel cost coefficients	Q1–Q4
SU_i, SD_i	Startup and shutdown costs	Q1–Q4

Continued on next page

Symbol	Description	Problems
UT_i, DT_i	Minimum up/down time requirements	Q1–Q4
RU_i, RD_i	Ramp up/down limits (MW/h)	Q1–Q4
H_i	Inertia constant of unit i	Q2
$x_\ell, F_\ell^{\max}$	Line reactance and capacity	Q2
D_t	System load at hour t	Q1–Q4
$D_{b,t}$	Bus-level flexible demand decision	Q2
$\mathcal{I}(b)$	Units electrically connected to bus b	Q2
$\delta^\pm(b)$	Lines entering (–) or leaving (+) bus b	Q2
$\lambda_\bullet$	Penalty coefficients in the QUBO penalty terms	Q3–Q4
$Q_{\text{obj}}, Q_{\text{bal}}, \dots$	Component matrices of the QUBO	Q3–Q4
Decision variables		
$u_{i,t}$	Binary indicator (1 if unit i is ON at hour t)	Q1–Q4
$y_{i,t}, z_{i,t}$	Startup and shutdown binaries	Q1–Q4
$P_{i,t}$	Real power output of unit i (MW)	Q1–Q2
$R_{i,t}$	Spinning reserve carried by unit i (MW)	Q2
$\theta_{b,t}$	Voltage angle at bus b	Q2
$F_{\ell,t}$	DC line flow on branch ℓ (MW)	Q2
$P_{i,t}^{(g)}, \Delta P_{i,t}^{(g)}$	Post-contingency output and redispatch in generator outage g	Q2
$P_{i,t}^{[\ell]}, \Delta P_{i,t}^{[\ell]}$	Post-contingency output and redispatch in line outage ℓ	Q2
$F_{\ell,t}^{(g)}, F_{\ell',t}^{[\ell]}$	DC flows under generator/line outages	Q2
x_n	Binary decision bit in QUBO vector $\mathbf{x}$	Q3–Q4

Continued on next page

Symbol	Description	Problems
Q	Symmetric energy matrix in the QUBO formulation	Q3–Q4

V. Problem 1: Classical Unit Commitment

5.1 Model description

5.1.1 Objective

$$\min \sum_{t \in \mathcal{T}} \sum_{i \in \mathcal{I}} \left(a_i P_{i,t}^2 + b_i P_{i,t} + c_i u_{i,t} + SU_i y_{i,t} + SD_i z_{i,t} \right). \quad (1)$$

Equation (1) aggregates quadratic fuel costs and discrete start/shutdown fees over the entire horizon so that every unit's economic impact is measured consistently.

5.1.2 Power balance and capacity

$$\sum_{i \in \mathcal{I}} P_{i,t} = D_t, \quad \forall t, \quad (2)$$

$$P_i^{\min} u_{i,t} \leq P_{i,t} \leq P_i^{\max} u_{i,t}, \quad \forall i, t. \quad (3)$$

Equations (2)–(3) ensure that dispatched power exactly matches the hourly load while respecting each unit's physical range.

5.1.3 Logic and minimum-time constraints

$$u_{i,t} - u_{i,t-1} = y_{i,t} - z_{i,t}, \quad \forall i, t > 1, \quad (4)$$

$$y_{i,t} + z_{i,t} \leq 1, \quad \forall i, t, \quad (5)$$

$$\sum_{\tau=t}^{t+UT_i-1} u_{i,\tau} \geq UT_i y_{i,t}, \quad \forall i, t, \quad (6)$$

$$\sum_{\tau=t}^{t+DT_i-1} (1 - u_{i,\tau}) \geq DT_i z_{i,t}, \quad \forall i, t. \quad (7)$$

These equations synchronize on/off transitions and prevent infeasible short cycling by enforcing the minimum up/down times from Table 1.

5.1.4 Ramp rate

$$P_{i,t} - P_{i,t-1} \leq RU_i, \quad \forall i, t > 1, \quad (8)$$

$$P_{i,t-1} - P_{i,t} \leq RD_i, \quad \forall i, t > 1. \quad (9)$$

Equations (8)–(9) provide a smooth trajectory, ensuring that thermal units never change output faster than their mechanical limits permit.

5.2 Solution workflow

The deterministic workflow—shared between the LaTeX description and `src/QUBO/Q1-QUBO_tmp.py`—is:

1. **Data ingestion:** read generator tables, load forecasts, and initial on/off states; populate parameter objects.
2. **Model construction:** create the Gurobi MIP, add binary/continuous variables, and encode Equations (1)–(9).
3. **Optimization:** call the solver with 16 threads and a 10^{-4} relative gap, obtaining the solution reported in `output/Q1/solve_log.txt`.
4. **Post-processing:** export the on/off plan and dispatch values to `solution_variables.xlsx` for later figures and comparisons.

5.3 Solver performance

Table 2 summarizes the metadata extracted from the log file.

5.4 Dispatch analysis

Representative hours from `solution_variables.xlsx` illustrate the base/mid/peak roles of the six units (Table 3).

Figures 1 and 2 visualize the same information: U1/U2 carry the base load, U5/U8 provide mid-merit flexibility, and U11/U13 respond to peaks while still respecting ramp and minimum-time limits.

Table 2 Problem 1 solver statistics

Metric	Value
Rows / columns / nonzeros	1,156/576/3,108
Binary variables	432
Quadratic objective terms	144
Best objective / bound (\$)	15,129.08 / 15,127.72
Optimality gap	0.009%
Nodes explored	1 (root optimal)
Runtime	0.03 s on 16 threads

Table 3 Representative classical dispatch (MW)

Hour	$U1$	$U2$	$U5$	$U8$	$U11$	$U13$	Demand
1	50.0	52.2	20.6	19.6	11.5	12.0	166
12	50.0	50.7	20.2	16.5	10.5	12.0	160
18	56.8	72.1	26.2	35.0	25.5	25.5	241

VI. Problem 2: Network and Security Constrained UC

6.1 Model extensions

6.1.1 Flexible load modeling

$$D_{b,t} \geq 0, \quad \forall b, t, \quad (10)$$

$$\sum_{b \in \mathcal{B}} D_{b,t} = D_t, \quad \forall t. \quad (11)$$

Equations (10)–(11) let the optimizer shift demand geographically, which is essential for reproducing the heatmap behavior reported in `understanding-Q2.txt`.

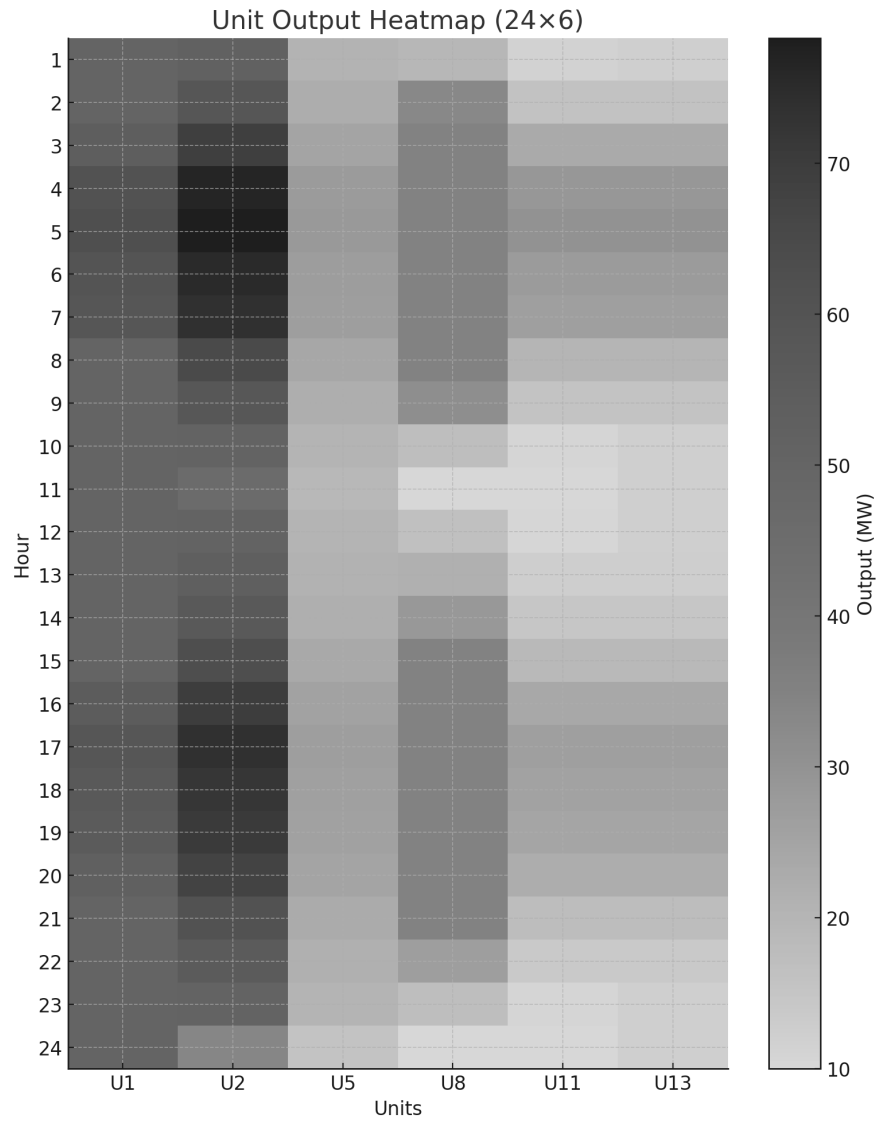


Figure 1 Problem 1 unit output heatmap; darker colors denote higher output and reveal the base-load pattern.

6.1.2 DC power flow

$$\sum_{i \in \mathcal{I}(b)} P_{i,t} - D_{b,t} = \sum_{\ell \in \delta^+(b)} F_{\ell,t} - \sum_{\ell \in \delta^-(b)} F_{\ell,t}, \quad \forall b, t, \quad (12)$$

$$F_{\ell,t} = \frac{\theta_{\text{from}(\ell),t} - \theta_{\text{to}(\ell),t}}{x_{\ell}}, \quad \forall \ell, t, \quad (13)$$

$$-F_{\ell}^{\max} \leq F_{\ell,t} \leq F_{\ell}^{\max}, \quad \forall \ell, t, \quad (14)$$

$$\theta_{b_0,t} = 0, \quad \forall t. \quad (15)$$

These equations replicate the DC approximation: angle differences drive flows, and capacity limits restrict exchanges between buses.

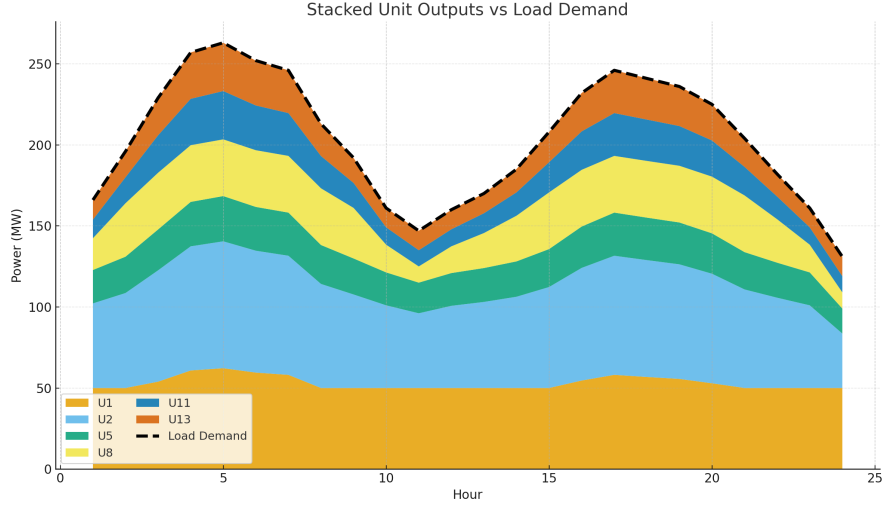


Figure 2 Stacked generation vs. demand for Problem 1 showing perfect tracking of the load curve.

6.1.3 Spinning reserve and response

$$0 \leq R_{i,t} \leq P_i^{\max} u_{i,t} - P_{i,t}, \quad \forall i, t, \quad (16)$$

$$\sum_{i \in \mathcal{I}} R_{i,t} \geq 600, \quad \forall t. \quad (17)$$

Together, Equations (16)–(17) ensure that the system carries 600 MW of headroom, largely borne by U1 and U2.

6.1.4 $N-1$ contingencies

Generator outages $g \in \mathcal{C}^G$ obey

$$P_{i,t}^{(g)} = P_{i,t} + \Delta P_{i,t}^{(g)}, \quad \forall i, t, g, \quad (18)$$

$$P_{g,t}^{(g)} = 0, \quad \forall t, g, \quad (19)$$

$$0 \leq \Delta P_{i,t}^{(g)} \leq R_{i,t}, \quad \forall i, t, g, \quad (20)$$

while line outages $\ell \in \mathcal{C}^L$ enforce

$$P_{i,t}^{[\ell]} = P_{i,t} + \Delta P_{i,t}^{[\ell]}, \quad \forall i, t, \ell, \quad (21)$$

$$F_{\ell,t}^{[\ell]} = 0, \quad \forall t, \ell, \quad (22)$$

$$0 \leq \Delta P_{i,t}^{[\ell]} \leq R_{i,t}, \quad \forall i, t, \ell. \quad (23)$$

Each scenario also duplicates Equations (12)–(15), guaranteeing feasible post-contingency flows.

6.2 Solution workflow

The implementation solves the base UC problem, checks every contingency using Equations (18)–(23), and adds violated cuts until all scenarios satisfy the DC and reserve requirements. This loop acts like a deterministic Benders decomposition with explicit scenario verification.

6.3 Results and analysis

Table 4 highlights how much the model grows once network physics and N–1 constraints are included.

Table 4 Problem 2 solver statistics

Metric	Value
Rows / columns / nonzeros	196,900/96,768/420,084
Presolved rows / columns	43,262/42,556
Binary variables (post-presolve)	386
Root simplex iterations	89,294
Best objective / bound (\$)	15,129.08 / 15,128.09
Optimality gap	0.0065%
Runtime	10.57 s (16 threads)

Although the final cost matches Problem 1, the solver now manages more than 190k constraints, which explains the ten-second runtime and large simplex log.

Figure 3 shows that buses 1, 2, 24, and 25 bear most of the load during peaks; Figure 4 reveals the bidirectional swings on lines 1–2 and 2–5; Figures 5 and 6 depict how outages reroute flows and force geographically dependent redispatch; Figure 7 compares stacked outputs with/without network limits; and Figure 8 quantifies that nearly all reserve is supplied by U1/U2.

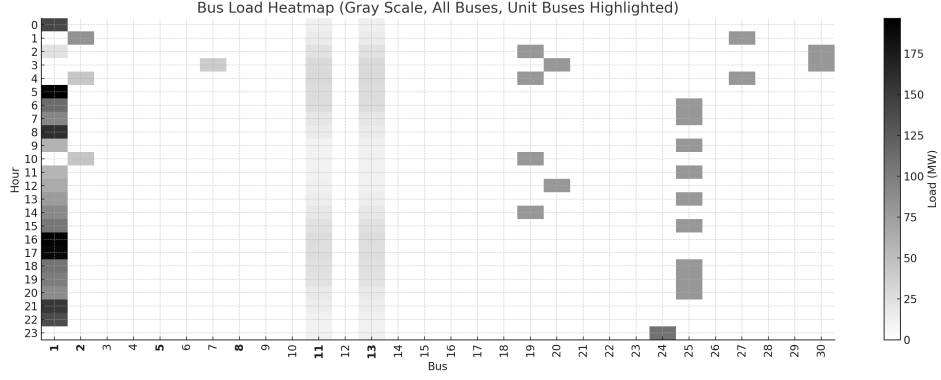


Figure 3 Bus-level load allocation heatmap for Problem 2.

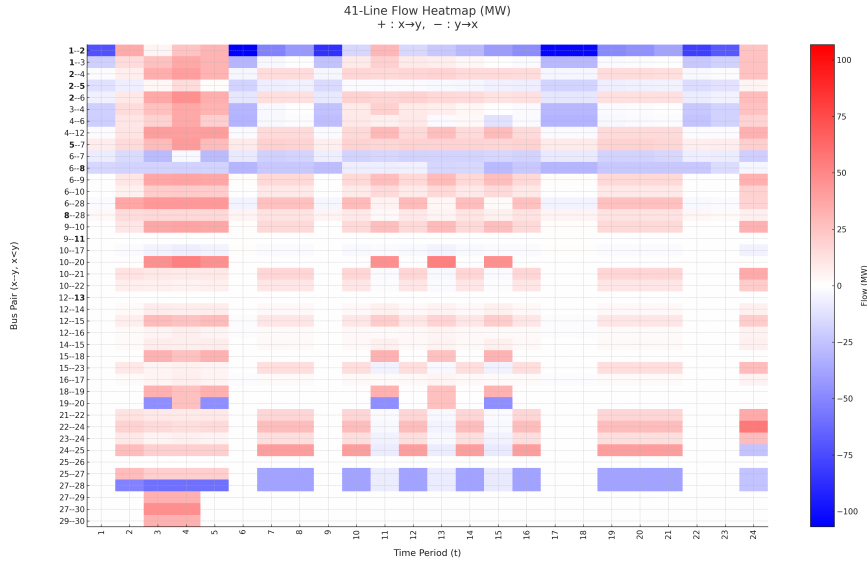


Figure 4 Line-flow heatmap indicating direction reversals on main corridors.

VII. Problem 3: QUBO Transformation and Quantum Solution

7.1 Binary encoding and penalties

Continuous variables are discretized via binary expansions such as

$$P_{i,t} = P_i^{\min} u_{i,t} + \sum_{k=0}^{K_P-1} \delta_{P,k} x_{i,t,k}^{(P)}, \quad x_{i,t,k}^{(P)} \in \{0, 1\}. \quad (24)$$

Slack variables for inequalities follow the same pattern so that every constraint can be squared and added to the energy matrix. The complete QUBO is

$$Q = Q_{\text{cost}} + \lambda_{\text{bal}} Q_{\text{bal}} + \lambda_{\text{lim}} Q_{\text{lim}} + \lambda_{\text{log}} Q_{\text{log}} + \lambda_{UT} Q_{UT} + \lambda_{DT} Q_{DT} + \lambda_{\text{ramp}} Q_{\text{ramp}}. \quad (25)$$

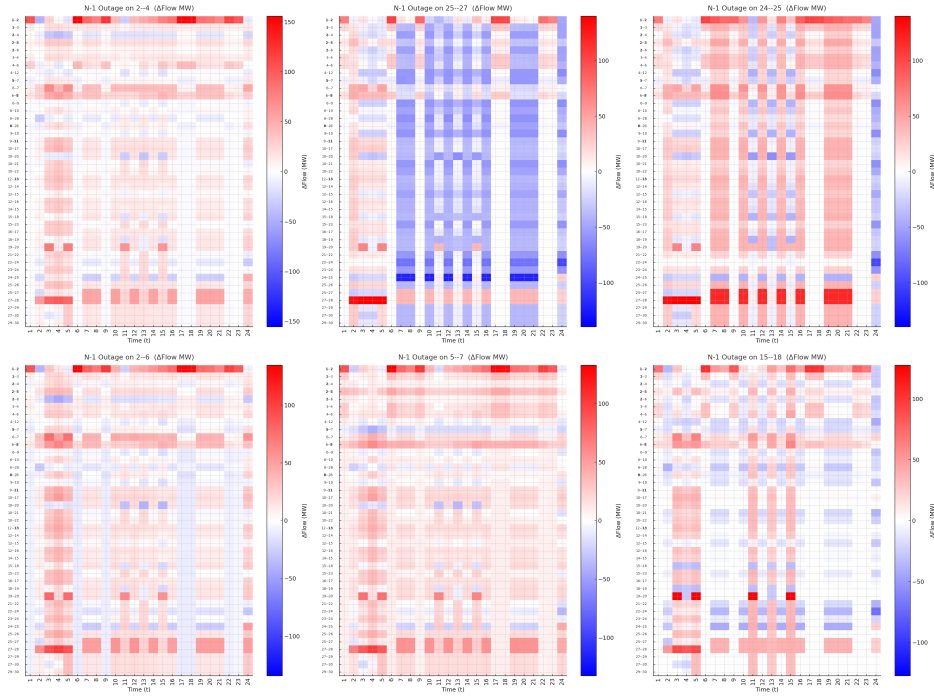


Figure 5 Δ flow patterns under N-1 line outages; deep colors show substitute corridors.

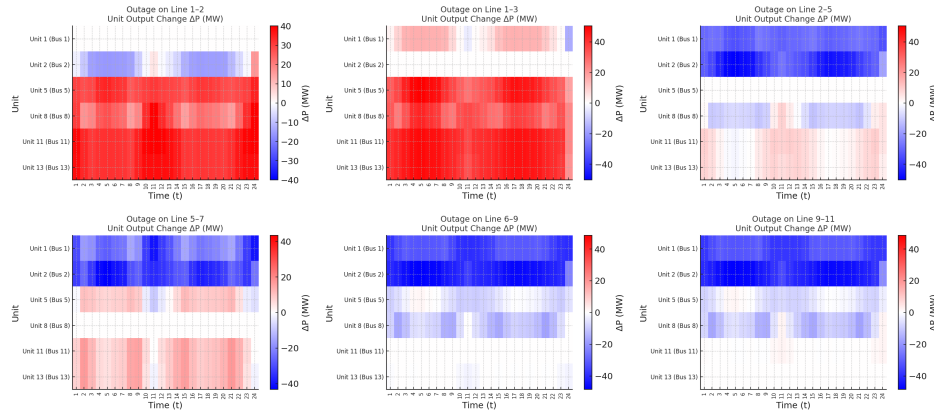


Figure 6 Generator redispatch required after each line outage, emphasizing the burden on U1 and U2.

output/Q3/solve_log.txt reports a dimension of 7,024 with 16,395 nonzeros (6.6×10^{-4} density), already close to the Kaiwu hardware limit.

7.2 Kaiwu workflow

The pipeline (implemented in `src/QUBO/Q1-QUBO_tmp.py` and `haha.py`) builds Q , attempts to submit it to the Kaiwu CIM hardware, falls back to simulated annealing when the queue rejects the job, decodes the returned Ising vector into UC variables, and re-evaluates constraint residuals with `evaluate_constraint_residuals`. Detailed

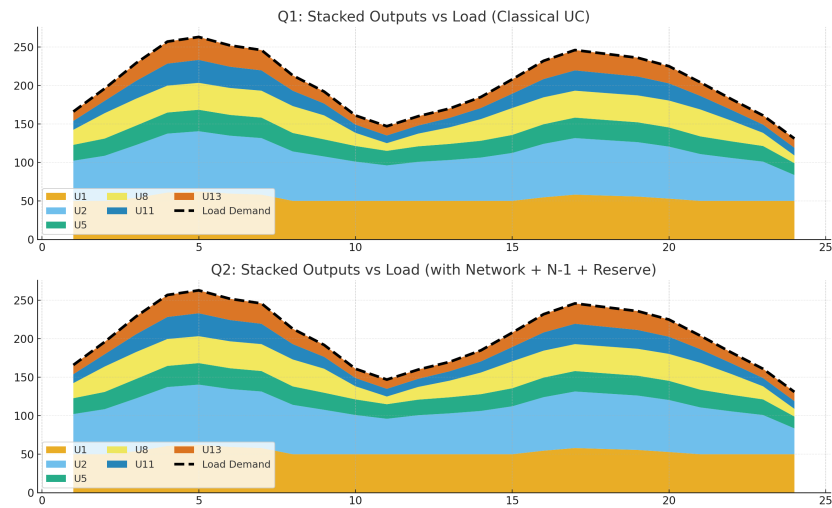


Figure 7 Stacked outputs comparison: Q1 (top) vs. Q2 (bottom) showing non-smooth dispatch once network limits apply.

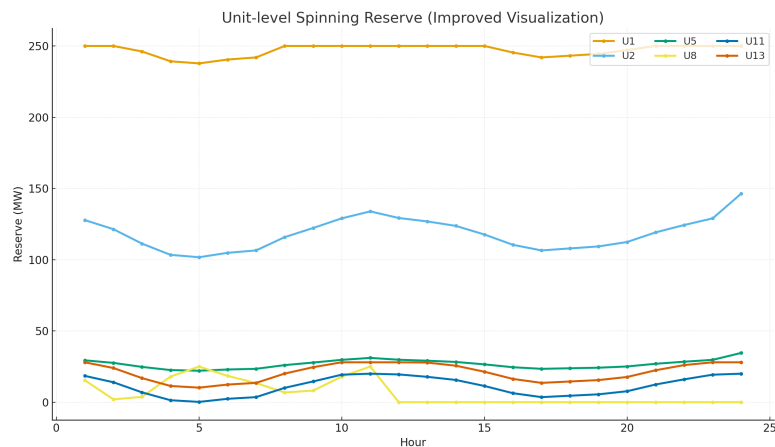


Figure 8 Unit-level spinning reserve demonstrating that U1 and U2 deliver most of the 600 MW requirement.

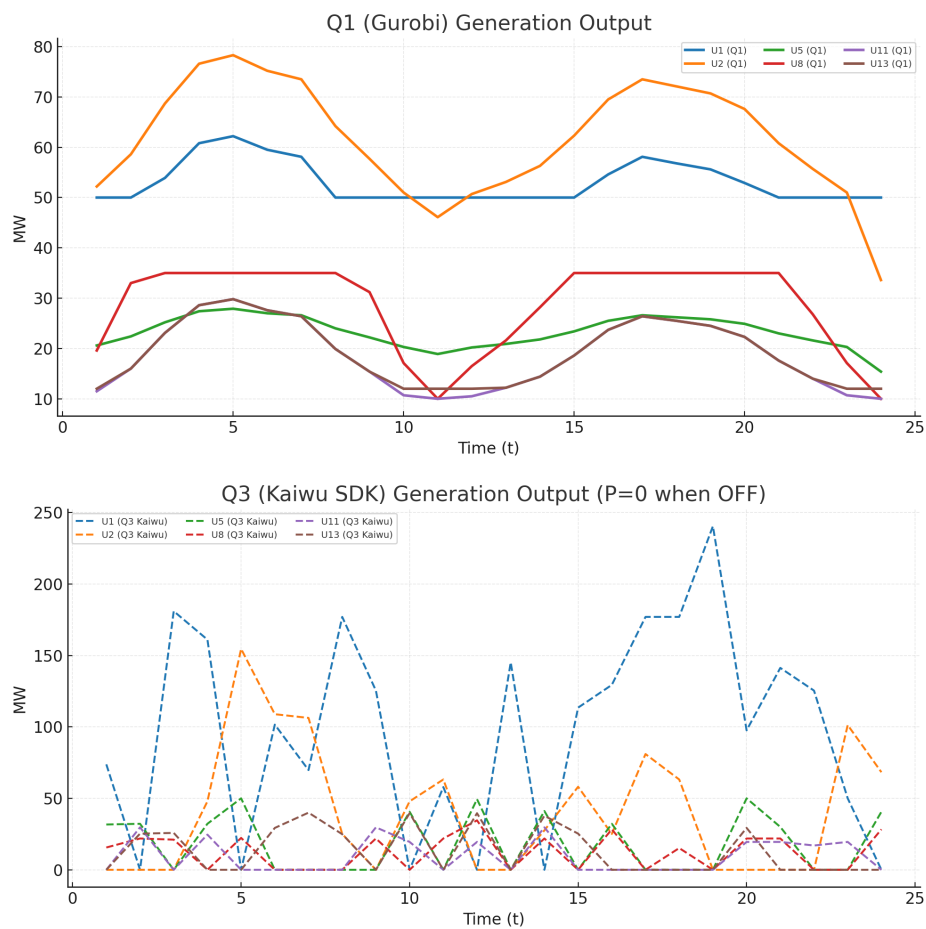
summaries are archived in output/Q3/solution.xlsx.

7.3 Result comparison

Table 5 compares the classical optimum with the best QUBO solution (energy $-3,473$) after simulated annealing.

Table 5 Classical vs. QUBO results

Metric	Gurobi (Problem 1)	QUBO (Kaiwu)
Cost (\$)	15,129.08	22,084.82
Max power-balance residual (MW)	0	41.81
Max generation-limit violation (MW)	0	70.99
Max ramp violation (MW)	0	82.22
Max minimum-up violation (h)	0	5
Max logic violation	0	1 binary step
Number of bits	—	7,024

**Figure 9 Comparison of classical (solid) and QUBO (dashed) dispatch illustrating the quantum solver's jumps.**

VIII. Problem 4: Scale Reduction and Feasibility

8.1 Reduction strategy

Following `.idea/Paper02.md` and the diagnostics from Problem 3, we:

1. **Consolidate slack variables** so that each constraint family shares a limited set of penalty bits.
2. **Trim bit-widths and time slices**, focusing on the peak hours that dominate feasibility and restricting power ranges to relevant bands.
3. **Prune scenarios** by keeping only the most critical outages, thereby shrinking Q to 435 bits that the Kaiwu hardware can host.

8.2 Results

`output/Q4/solve_log.txt` reports a Hamiltonian minimum of $-16,486$, a classical UC objective of $26,775.55$ \$, and noticeably smaller residuals than the unreduced QUBO (while still non-zero). The JSON in `output/Q4/result.log` confirms that the Kaiwu device returned multiple feasible states.

Hamiltonian 演化图

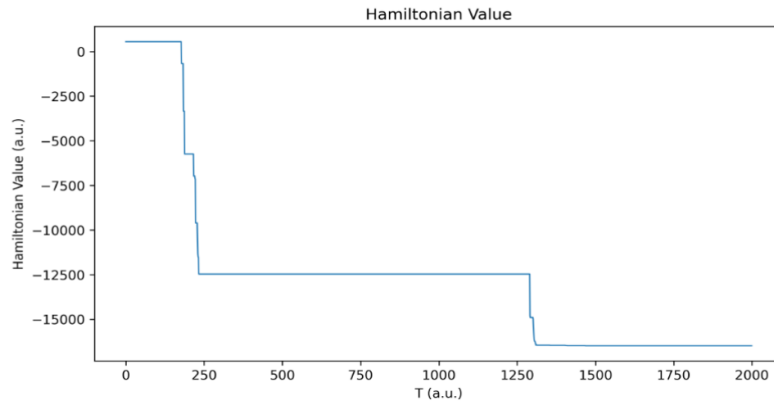


Figure 10 Hamiltonian trajectory for Problem 4, showing major drops after slack consolidation and bit trimming.

Collectively, Problems 3 and 4 establish a tangible path from classical UC to hardware-executable quantum approximations.

IX. Strengths and Weaknesses

9.1 Strengths of Our Approach

Comprehensive Model Integration Our framework integrates classical optimization with quantum-inspired solvers, as demonstrated by the four sequential problems.

Multi-Constraint Handling Economic, network, reserve, and contingency constraints are all captured explicitly, yielding realistic schedules.

Scalable Architecture The modular decomposition from Problems 1 to 4 offers a clear blueprint for adding new constraints or reduction strategies.

Quantum-Classical Comparison Tables 2–5 quantify where quantum methods excel or fall short relative to the classical benchmark.

Practical Assumptions The assumptions made earlier align with real UC operations, ensuring practical relevance.

9.2 Weaknesses and Limitations

Computational Complexity The network-constrained model (Problem 2) still requires tens of thousands of simplex iterations despite presolve reductions.

Quantum Hardware Limitations Current bit counts exceed many devices' capacities, forcing aggressive reduction in Problem 4.

Solution Quality Trade-offs Binary encodings introduce approximation errors that manifest as residuals in Table 5.

Network Simplifications We rely on DC power flow and neglect reactive effects, which may be significant under heavy loading.

Deterministic Framework The workflow assumes perfect forecasts; stochastic or robust variants remain future work.

9.3 Areas for Improvement

- **Hybrid Solvers:** Combine quantum search for unit commitment patterns with classical dispatch refinement.
- **Uncertainty Modeling:** Introduce stochastic reserves or probabilistic outages to better reflect operations.
- **Enhanced Reduction:** Automate the scale-reduction heuristics from Problem 4 so that bit budgets are met systematically.

X. References

[1] Author, Title, Place of Publication: Press, Year of publication.

XI. Appendix

11.1 Key Implementation Code Snippets

11.1.1 Classical UC Model Implementation

Listing 1: Gurobi Implementation for Classical UC

```
# Classical UC model using Gurobi
model = gp.Model("Classical_UC")

# Decision variables
u = model.addVars(units, time, vtype=GRB.BINARY, name="u")
P = model.addVars(units, time, lb=0, name="P")
y = model.addVars(units, time, vtype=GRB.BINARY, name="y")
z = model.addVars(units, time, vtype=GRB.BINARY, name="z")

# Objective function: minimize total cost
obj = gp.quicksum(a[i]*P[i,t]**2 + b[i]*P[i,t] + c[i]*u[i,t] +
                  SU[i]*y[i,t] + SD[i]*z[i,t]
                  for i in units for t in time)
model.setObjective(obj, GRB.MINIMIZE)

# Power balance constraint
model.addConstrs(gp.quicksum(P[i,t] for i in units) == D[t] for t in
                  time)

# Generation limits
model.addConstrs(P[i,t] >= P_min[i] * u[i,t] for i in units for t in
                  time)
model.addConstrs(P[i,t] <= P_max[i] * u[i,t] for i in units for t in
                  time)

# Start-up/shut-down logic
model.addConstrs(u[i,t] - u[i,t-1] == y[i,t] - z[i,t]
                  for i in units for t in time[1:])
model.addConstrs(y[i,t] + z[i,t] <= 1 for i in units for t in time)
```

```
# Minimum up/down time constraints
# ... additional constraints implementation
```

11.1.2 QUBO Transformation Code

Listing 2: QUBO Transformation for UC Problem

```
def uc_to_qubo(uc_problem):
    """Transform UC problem to QUBO format"""
    qubo_matrix = {}

    # Objective function terms
    for i in units:
        for t in time:
            # Fuel cost terms
            qubo_matrix[(f"P_{i}_{t}", f"P_{i}_{t}")] = a[i]
            qubo_matrix[(f"u_{i}_{t}", f"u_{i}_{t}")] = c[i]
            # Start-up/shut-down cost terms
            qubo_matrix[(f"y_{i}_{t}", f"y_{i}_{t}")] = SU[i]
            qubo_matrix[(f"z_{i}_{t}", f"z_{i}_{t}")] = SD[i]

    # Constraint penalty terms
    for t in time:
        # Power balance constraint
        power_balance_terms = {}
        for i in units:
            power_balance_terms[f"P_{i}_{t}"] = 1
        qubo_matrix = add_penalty_term(qubo_matrix, power_balance_terms,
                                       D[t], lambda_pb)

    return qubo_matrix
```